UNIVERSITY
OF COLOGNE

WORKSHOP 02

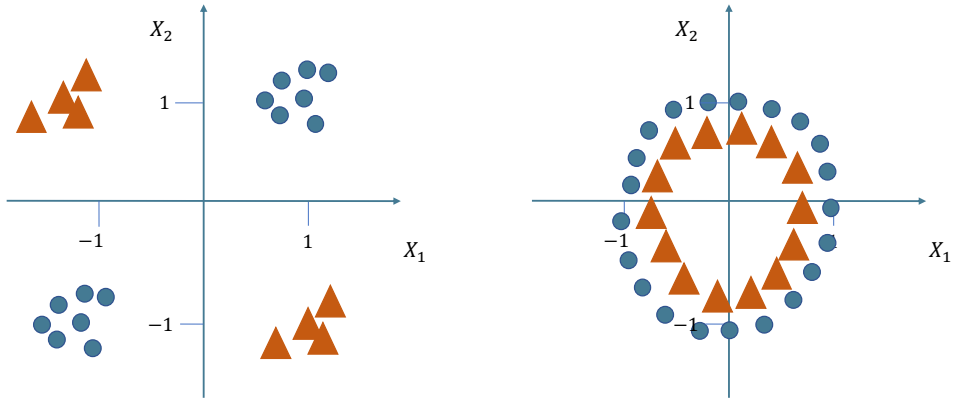
Advanced Analytics and Applications

Agenda

1	**Lecture Recap**
2	Exemplary Exam Questions
3	Coding Tasks
4	Team Assignment

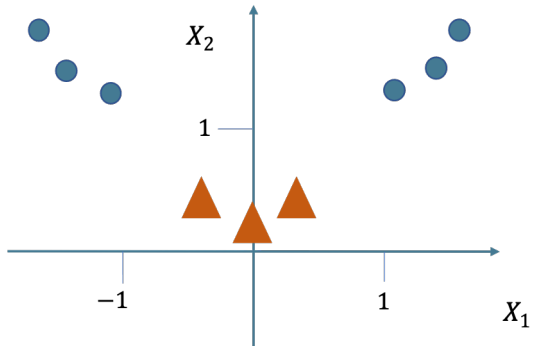
Kernels: The Motivation

- Sometimes **data isn't linearly separable** in its original space, making it difficult for linear classifiers to distinguish between classes effectively:



- For each distribution we could try and find a **projection** into a space where the data is linearly seperable:

- In this example, we can trivially find a polynomial projection $\phi(\mathrm{x}) = (\mathrm{x}, \mathrm{x}^2)$



Kernels: The Trick

- Sometimes, we could find a projection by simply looking at the data…
- It's impossible to always come up with a correct projection $\phi(x)$ manually, though!
- Good thing is we don't need to:
  Mercer's Theorem: For certain functions $\kappa(x_i, x_j)$ it is guaranteed that the dot product $\phi(x_i)^T\phi(x_j)$ simplifies to that function of just x_i and x_j.
- These **kernel functions** can be substituted in for every occurrence of dot products in all equations of the machine learning algorithm
- They provide similarity measures, like any dot product X^TX, just implicitly in higher space.

Why is the **dot product** so important?
Training: $w^* = (X^TX + \lambda I)^{-1}X^T\boldsymbol{y} \Rightarrow (\phi^T\phi + \lambda I)^{-1}\phi^T\boldsymbol{y}$
Prediction: $y(x) = w^T\phi(x)^1$

$$\underbrace{\boldsymbol{\phi}(\boldsymbol{x}_i)^\top\boldsymbol{\phi}(\boldsymbol{x}_j)}_{\text{Dot product in high-d space}} = \underbrace{\boldsymbol{\kappa}(\boldsymbol{x}_i, \boldsymbol{x}_j)}_{\text{Kernel function in original space}}$$

Kernels: Finding suitable $\kappa(\boldsymbol{x_i}, \boldsymbol{x_j})$

- How do we find kernel functions $\kappa(\boldsymbol{x_i}, \boldsymbol{x_j})$ for which Mercer's Theorem holds?
- Simple answer: We don't, mathematicians did.
- Longer answer: We test if the matrix $\boldsymbol{K}$ formed by evaluating the kernel on all pairs $(x_i, x_j) \in \{x_1, \dots, x_n\}$ is **positive semi-definite** ($x^T \mathrm{K} x \geq 0$ for any non-zero vector x):
  This is the **Gram Matrix**. Then, κ is a valid kernel!
- Even longer answer: People that see even less daylight than computer scientists (mathematicians) look outside machine learning and search for functions that are known to satisfy certain conditions* that make them a candidate for kernel functions (like Gaussians for the RBF kernel)

Several kernels are ready to be used, or a recombination of them:

- **Linear Kernel**: $\kappa(x_i, x_j) = x_i^T x_j$
  This is the Euclidean dot product, which is the cosine of the angle between the two vectors.
- **Polynomial Kernel**: $\kappa\left(x_i, x_j\right) = \left(\gamma x^T x_j + r\right)^d$
  Looks at similarity of polynomial combinations of x up to degree d
- **RBF Kernel**: $\kappa\left(x_i, x_j\right) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$
  Similarity measure between 0 (in the limit) and 1 (at $x_i = x_j$), with $\|x_i - x_j\|^2$ being the squared Euclidean distance

* Positive definite, symmetric, continuous, translation-invariant, its Fourier transformation is non-negative.

Lecture Slide

Kernels: Example (backwards)

- If we would know, in a perfect world, a projection for our data:

$$\phi: x \to \phi(x) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$$

$$\phi(x_m)^T\phi(x_n) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}^T \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}$$

$$\phi(x_m)^T\phi(x_n) = x_{m1}^2x_{n1}^2 + 2x_{m1}x_{m2}x_{n1}x_{n2} + x_{m2}^2x_{n2}^2$$

...

$$\phi(x_m)^T\phi(x_n) = \left(\mathbf{x}_m^T\boldsymbol{x}_n\right)^2$$

- We just showed, that it is exactly the polynomial kernel (with $y = 1, r = 0$):

$$\kappa\,(x, x') = (\gamma x^T x' + r)^d$$

- We just showed that we can do any regression with polynomial kernel without needing to know the complex basis vector $\phi: x \to \phi(x)$!
- And this applies to any kernel function!

Kernels: The Math

- Why are we doing all the math in the lecture and working with matrix notation?
- This is to show that

1. We can reformulate the **training objective** from

$$\ell(w) = \sum_{i=1}^{N} (w^T \phi(x_i) - y_i)^2 + \frac{\lambda}{2} w^T w$$

depends on the knowledge of ϕ

to

$$\ell(\boldsymbol{\alpha}) = \boldsymbol{\alpha}^T \boldsymbol{K} \boldsymbol{K} \boldsymbol{\alpha} - 2\boldsymbol{y}^T \boldsymbol{K} \boldsymbol{\alpha} + \boldsymbol{y}^T \boldsymbol{y} + \frac{\lambda}{2} \boldsymbol{\alpha}^T \boldsymbol{K} \boldsymbol{\alpha}$$

depends on the existence of $\boldsymbol{K}$

2. We can reformulate the **prediction equation** from

$$y(x) = w^T \phi(x)$$

depends on the knowledge of ϕ

to

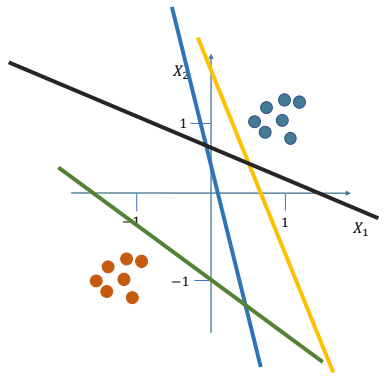
$$y(x) = \boldsymbol{y}^T (\boldsymbol{K} + \lambda \boldsymbol{I})^{-1} \kappa(x)$$

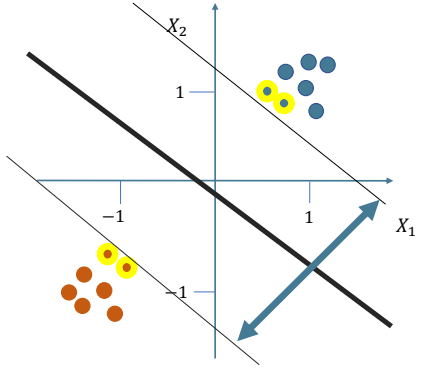
depends on the existence of $\boldsymbol{K}$

SVM: The Basics

- For linearly seperable data, an infinite number of hyperplanes exist that could separate the data. Which one do we pick?

- We want to use the one that is separates data “the most”, meaning it is the furthest away from any data point (what we call “support vectors”)





SVM: Finding the Hyperplane

(*) $min \frac{1}{2}||w||^2$ from the lecture is a convenience form for the math that is equivalent!

A hyperplane is defined by:

$$w^T x + b$$
$$\vec{n} = w\,, \vec{n} * -\vec{z} = b$$

For SVM we need to consider three hyperplanes:

- $\mathrm{H}_0 = \{x \in \mathbb{R}: w^T x + b = 0\}$
- $\mathrm{H}_1 = \{x \in \mathbb{R}: w^T x + b = 1\}$
- $\mathrm{H}_2 = \{x \in \mathbb{R}: w^T x + b = -1\}$

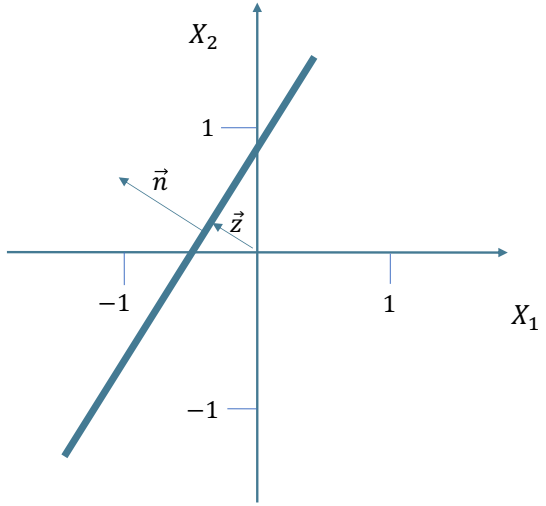
Optimization Problem

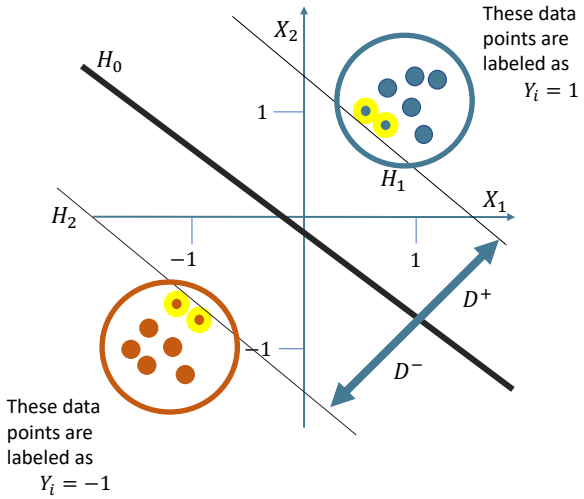
To maximize the margin $\frac{2}{||w||}$ we

$$\min ||w|| \quad (*)$$

There shall not be data points between H_1 and H_2

$$s.t.. \quad y_i\,(x_i * w) \geq 1$$





SVM: Using Kernels

- What do we need for kernels to be applicable to a machine learning algorithm?
- Data points $\boldsymbol{x_i}$ **only need to appear inside dot products** $\boldsymbol{(x_i^T x_j)}$ because then we can use kernels to replace the otherwise needed dot product of projections $\phi(x_i)^T \phi(x_j)$.
- In our current formulation of the problem,

$$\min \cdot \quad \frac{1}{2}||w||^2$$

$$s.t. \quad y_i\,(x_i * w) \geq 1$$

- x is everywhere, but not in dot products exclusively…

SVM: Using Kernels

- Luckily for us, the dual form of the optimization problem looks different:

$$\max \quad L_D(a_i) = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j y_i y_j \boldsymbol{x_i x_j} + \sum_{i=1}^{N} a_i$$

$$s.t. \qquad a_i \geq 0; \sum_{i=1}^{N} a_i y_i \geq 0$$

- Here, x_i is only found in dot products, and we can substitute in kernels!

$$\max L_D(a_i) = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j y_i y_j \boldsymbol{\kappa}(\boldsymbol{x_i}, \boldsymbol{x_j}) + \sum_{i=1}^{N} a_i$$

SVM: The (not-relevant) math

- How do we get there:
- For constrained optimization problems, Langrangian Multipliers (a) are a regular method for replacing constraints (one a_i for every constraint) into the objective:

$$\min L_p = \frac{1}{2}||w||^2 - \sum_{i=1}^{N} a_i y_i (w^T x_i + b) + \sum_{i=1}^{N} a_i$$

Questions that regularly arise

- **Question:** “What is the difference between a linear model and a linear kernel”
- **Answer:** Those are mathematically hard to compare as there doesn’t exist something like a non-kernelized “vanilla”/linear SVM version – the vanilla one essentially uses dot products, which IS a linear kernel.
- **Question:** “What is the dimensionality that kernels work in, generally?”
- **Answer:** A general claim cannot be made, but three things are certain:
  - 1. Linear kernels work in the input dimension.
  - 2. Polynomial kernels work in dimensions that grow by $O(n^d)$ (do binomial expansion to find out)
  - 3. RBF kernels mathematically work in infinite-dimensional space (do Taylor expansion to find out)

Agenda

1	Lecture Recap
2	**Exemplary Exam Questions**
3	Coding Tasks
4	Team Assignment

Question 1: Feature Transformation vs. Kernels

Consider two alternatives: Using a polynomial kernel and creating polynomial transformations on the features, a technique we can use to introduce non-linearity in $x \rightarrow y$ in linear models (like linear SVM or linear regression).

Polynomial features: $y = \theta_0 + \theta_1 x_1 + \theta_2 x_1 x_2 + \theta_3 x_2^2$

Polynomial kernel: $y(x) = \boldsymbol{y}^{\boldsymbol{T}} (\boldsymbol{K} + \lambda \boldsymbol{I})^{-\mathbf{1}} \kappa(x)$

$$\kappa(x, x') = (\gamma x^T x' + r)^d$$

a) What is the difference between these two?

b) Briefly explain an advantage (disadvantage of the other) of the two, each.

Question 1: Feature Transformation vs. Kernels

Consider two alternatives: Using a polynomial kernel and creating polynomial transformations on the features, a technique we can use to introduce non-linearity in $x \rightarrow y$ in linear models (like linear SVM or linear regression).

Polynomial features: $y = \theta_0 + \theta_1 x_1 + \theta_2 x_1 x_2 + \theta_3 x_2^2$

Polynomial kernel: $y(x) = \boldsymbol{y}^{\boldsymbol{T}} (\boldsymbol{K} + \lambda \boldsymbol{I})^{-\mathbf{1}} \kappa(x)$

$$\kappa(x, x') = (\gamma x^T x' + r)^d$$

a) What is the difference between these two?
With polynomial features, the complexity is apparent in the features and coefficients correspond

b) Briefly explain an advantage (disadvantage of the other) of the two, each.
Advantage Kernels (Disadvantage Feature Transformation): They do not introduce higher dimensionality in the features! (some in the functions, but less so)
Advantage Feature Transformation (Disadvantage Kernels): They estimate interpretable coefficients for each transformed feature, so you will be able to infer feature importance from that more easily.

Multiple Choice Questions: SVM

Question: In practice, selecting the most appropriate kernel function (e.g., Linear, RBF, Polynomial) for an SVM applied to a specific dataset usually involves:

- ❑ Always choosing the RBF kernel as it implicitly maps to an infinite-dimensional space.
- ❑ Selecting the kernel based purely on the number of features in the original dataset (e.g., linear for high dimensions, RBF for low dimensions).
- ❑ Trying a few standard, promising kernels and evaluating their performance using techniques like cross-validation on the specific data.
- ❑ Choosing the kernel with the most complex mathematical formula to ensure the highest potential model flexibility.

Multiple Choice Questions: SVM

Question: In practice, selecting the most appropriate kernel function (e.g., Linear, RBF, Polynomial) for an SVM applied to a specific dataset usually involves:

- ☐ Always choosing the RBF kernel as it implicitly maps to an infinite-dimensional space.
- ☐ Selecting the kernel based purely on the number of features in the original dataset (e.g., linear for high dimensions, RBF for low dimensions).
- ☑ **Trying a few standard, promising kernels and evaluating their performance using techniques like cross-validation on the specific data.**
- ☐ Choosing the kernel with the most complex mathematical formula to ensure the highest potential model flexibility.

Multiple Choice Questions: SVM

Question: A common practical challenge when applying Support Vector Machines (SVMs) to problems with multiple (k>2) classes is that:

- ❑ It requires the development of new kernel functions specifically designed for multi-class scenarios, different from those used in binary SVM.
- ❑ It requires the use of strategies like One-vs-Rest (OvR) or One-vs-One (OvO), which necessitate training multiple independent binary SVM classifiers (either k for OvR or k(k-1)/2 for OvO), which increases the overall complexity and time.
- ❑ All common multi-class SVM strategies inevitably lead to ambiguous regions where a data point is predicted to belong to multiple classes simultaneously.

Multiple Choice Questions: SVM

Question: A common practical challenge when applying Support Vector Machines (SVMs) to problems with multiple (k>2) classes is that:

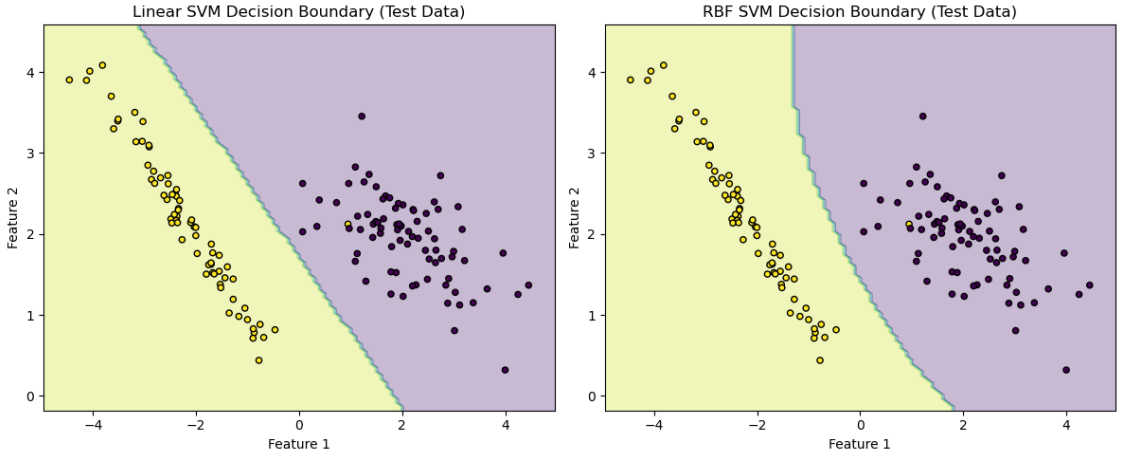
- ❑ It requires the development of new kernel functions specifically designed for multi-class scenarios, different from those used in binary SVM.
- ❑ **It requires the use of strategies like One-vs-Rest (OvR) or One-vs-One (OvO), which necessitate training multiple independent binary SVM classifiers (either k for OvR or k(k-1)/2 for OvO), which increases the overall complexity and time.**
- ❑ All common multi-class SVM strategies inevitably lead to ambiguous regions where a data point is predicted to belong to multiple classes simultaneously.

Agenda

1	Lecture Recap
2	Exemplary Exam Questions
3	**Coding Tasks**
4	Team Assignment

Questions that regularly arise

- **Question:** "Is there actually an example where linear SVM performs better than RBF?"
- **Answer:** Yes, if the definition is "strictly better than or equal". Then, linearly separable data distributions will lead to higher accuracy with linear kernel SVM. However, in practice, RBF will often perform close to that and thus make it hard to evaluate on "hand-drawn" data sets.
  One thing to keep in mind: Even if RBF performs equal, we bought that with a much higher complexity and thus computation time, meaning linear SVM would be the preferred choice.



Agenda

1	Lecture Recap
2	Exemplary Exam Questions
3	Coding Tasks
4	**Team Assignment**

Team Assignment

- We have 28 registrations, so we need to have **teams of four** students
- The team composition is uploaded in ILIAS, please organize yourselves accordingly!
- Remember to use the assignment template and organize all your work in a public GitHub repository!
- Let’s have a look into the tasks!

Contact

For general questions and enquiries on **research**, **teaching**, **job openings** and **new projects** refer to our website at www.is3.uni-koeln.de

For specific **enquiries regarding this course** contact us by sending an email to the **IS3 teaching address** at is3-teaching@wiso.uni-koeln.de

To help us process your request efficiently, please use the following subject line format:

[AAA] <Your request subject>